
Academic Events API

Relatorio Tecnico – Trabalho Final

Modulo 3: Desenvolvimento em C#

Disciplina: Desenvolvimento em C# – Modulo 3

Grupo: Grupo 6

Data: Junho de 2026

Integrantes do Grupo 6

1. Thailsson Clementino de Andrade
2. Stevao Whinter Marques de Andrade
3. Marcio Franklin de Oliveira Lima
4. Allef Oliveira Ramos
5. Juliana Ballin Lima

Sumário

1	Introducao	2
2	Problema e Solucao	2
2.1	O Problema	2
2.2	Nossa Solucao	2
3	Tecnologias Utilizadas	2
4	Arquitetura do Sistema	3
4.1	Visao Geral – Clean Architecture em Camadas	3
4.2	Fluxo de Dependencias entre Projetos	3
4.3	Por que essa Arquitetura?	3
5	Modelagem do Dominio	3
5.1	Entidades	3
5.2	Enumeradores	4
6	Autenticacao e Autorizacao	4
6.1	JWT Bearer Token	4
6.2	Configuracao do Token	4
6.3	Hash de Senha	5
7	Endpoints da API	5
7.1	Autenticacao (publico)	5
7.2	Eventos	5
7.3	Inscricoes, Comentarios e Reacoes	5
8	Camada de Infrastructure	6
8.1	DbContext e Configuracoes EF Core	6
8.2	Repository Pattern	6
8.3	Banco de Dados via Docker	6
9	Regras de Negocio Implementadas	6
10	Dificuldades e Deciso es Tecnicas	7
10.1	Versoes dos Pacotes NuGet	7
10.2	Forbid() vs StatusCode(403)	7
10.3	Application nao depende de Infrastructure	7
10.4	Validacao em Dois Niveis	7
11	Como Executar o Projeto	7
12	Conclusao	8

1 Introducao

O presente relatorio descreve o desenvolvimento da **Academic Events API**, uma API REST criada como trabalho final do Modulo 3 do curso de Desenvolvimento em C#. O projeto implementa um sistema de gerenciamento de eventos academicos – palestras, workshops e seminarios – seguindo as boas praticas de desenvolvimento com ASP.NET Core 8.

O objetivo principal foi aplicar os conceitos ensinados ao longo do modulo: orientacao a objetos, DTOs, services, repositories, acesso a banco de dados, autenticacao e autorizacao, e organizacao profissional de codigo em camadas separadas.

A plataforma permite que usuarios se cadastrem, criem e gerenciem eventos academicos, realizem inscricoes, publiquem comentarios e expressem reacoes a conteudos de interesse, funcionando de forma analoga a uma rede social voltada para o ambiente academico.

2 Problema e Solucao

2.1 O Problema

Eventos academicos – palestras, workshops, minicursos e seminarios – sao parte essencial da vida universitaria. No entanto, a organizacao e divulgacao desses eventos muitas vezes acontece de forma fragmentada: grupos de WhatsApp, cartazes fisicos e e-mails avulsos dificultam o acompanhamento das oportunidades disponiveis e o controle de inscricoes.

2.2 Nossa Solucao

A Academic Events API centraliza o ciclo de vida completo de um evento academico em uma unica plataforma, expondo endpoints REST documentados via Swagger. Qualquer aplicacao cliente – web, mobile ou desktop – pode consumir a API para:

- Listar e filtrar eventos por status ou organizador
- Criar e gerenciar eventos (organizadores autenticados)
- Realizar e acompanhar inscricoes
- Comentar e reagir a eventos de interesse

3 Tecnologias Utilizadas

Tecnologia	Versao	Finalidade
ASP.NET Core	8.0	Framework principal da API REST
Entity Framework Core	8.0.11	ORM para acesso ao banco de dados
Npgsql (PostgreSQL)	8.0.11	Driver de conexao com PostgreSQL
JWT Bearer Token	8.0.15	Autenticacao e autorizacao via token
BCrypt.Net-Next	4.2.0	Hash seguro de senhas dos usuarios
Swashbuckle / Swagger	6.6.2	Documentacao automatica dos endpoints
PostgreSQL	15	Banco de dados relacional (Docker)
Docker Compose	–	Ambiente de banco de dados isolado

4 Arquitetura do Sistema

4.1 Visao Geral – Clean Architecture em Camadas

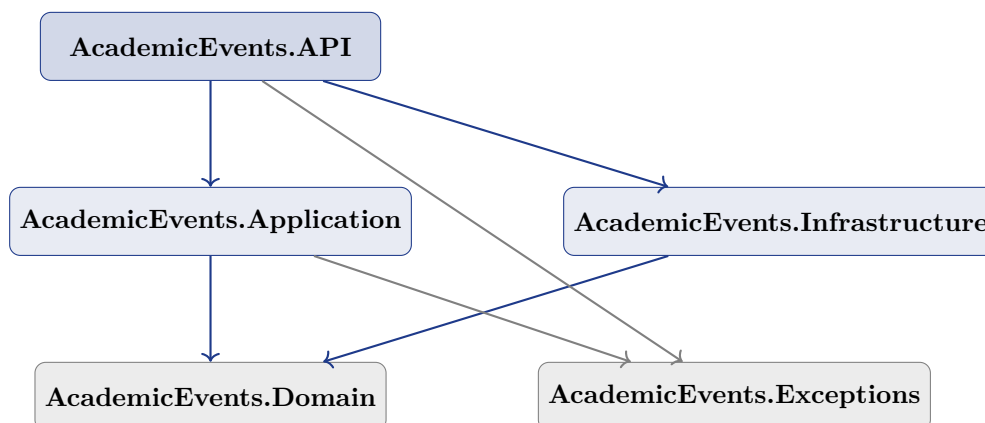
O projeto e organizado em cinco projetos C# separados dentro de uma unica solution, seguindo os principios de separacao de responsabilidades e inversao de dependencia:

Estrutura da Solution

- **AcademicEvents.Domain** – Entidades, enums, interfaces de repository (sem dependencias externas)
- **AcademicEvents.Exceptions** – Excecoes customizadas com semantica de negocio
- **AcademicEvents.Application** – DTOs, interfaces de service, implementacoes de service
- **AcademicEvents.Infrastructure** – DbContext, EF Core, implementacoes de repository
- **AcademicEvents.API** – Controllers, JWT, Swagger, Program.cs (ponto de entrada)

4.2 Fluxo de Dependencias entre Projetos

O grafico abaixo ilustra como os projetos se relacionam, respeitando a regra de que as camadas internas nunca dependem das externas:



4.3 Por que essa Arquitetura?

- **Domain sem dependencias:** as entidades e interfaces ficam isoladas, podendo ser testadas sem infraestrutura
- **Application so depende de Domain:** os services nao sabem como os dados sao persistidos – apenas chamam interfaces
- **Infrastructure implementa interfaces do Domain:** a inversao de dependencia permite trocar o banco (ex: SQLite para testes) sem mudar a logica de negocio
- **API orquestra tudo via DI:** o Program.cs conecta as implementacoes concretas usando extension methods de injecao de dependencia

5 Modelagem do Dominio

5.1 Entidades

O dominio e composto por cinco entidades principais com os respectivos relacionamentos:

Entidade	Campos principais	Descricao
User	Id, Nome, Email, SenhaHash, CriadoEm	Usuario da plataforma. Pode organizar eventos, se inscrever, comentar e reagir
Event	Id, Titulo, Descricao, DataInicio, DataFim, Local, Status, OrganizadorId	Evento academico criado por um organizador. Ciclo: Rascunho, Publicado, Cancelado, Concluido
Registration	Id, UsuarioId, EventoId, Status	Inscricao de um usuario em um evento. Indice unico impede duplicatas
Comment	Id, UsuarioId, EventoId, Conteudo	Comentario do usuario em um evento
Reaction	Id, UsuarioId, EventoId, Tipo	Reacao (curtida) de um usuario a um evento. Um usuario so pode ter uma reacao por evento

5.2 Enumeradores

Enum	Valores
StatusEvento	Rascunho, Publicado, Cancelado, Concluido
StatusInscricao	Pendente, Confirmada, Cancelada
TipoReacao	Curtir, Adorei, Interessante, VouParticipar

6 Autenticacao e Autorizacao

6.1 JWT Bearer Token

A autenticacao utiliza o padrao **JWT (JSON Web Token)** com algoritmo HMAC-SHA256. O fluxo e:

1. Usuario envia email e senha para POST /api/auth/login
2. O AuthService valida as credenciais com BCrypt
3. Um token JWT e gerado com as *claims*: `NameIdentifier` (id), `Email` e `Name`
4. O token e retornado e deve ser enviado no header `Authorization: Bearer {token}`
5. O middleware do ASP.NET valida o token automaticamente em todas as rotas com `[Authorize]`

6.2 Configuracao do Token

```
// appsettings.json
"Jwt": {
  "Key": "chave-secreta-minimo-32-caracteres-aqui",
  "Issuer": "AcademicEventsAPI",
  "Audience": "AcademicEventsClientes",
  "ExpiresInHours": 8
}
```

6.3 Hash de Senha

As senhas dos usuarios nunca sao armazenadas em texto puro. O **BCrypt** e usado para gerar um hash com salt automatico, tornando ataques de dicionario impraticaveis mesmo que o banco seja comprometido.

7 Endpoints da API

7.1 Autenticacao (publico)

Metodo	Rota	Retorno	Descricao
POST	/api/auth/register	200, 400	Cadastra usuario e retorna JWT
POST	/api/auth/login	200, 401	Autentica usuario e retorna JWT
GET	/api/me	200, 401	Dados do usuario autenticado (JWT)

7.2 Eventos

Metodo	Rota	Auth	Descricao
GET	/api/events	Nao	Lista todos os eventos
GET	/api/events?status=X	Nao	Filtra por status
GET	/api/events/{id}	Nao	Busca evento por id
GET	/api/events/meus	Sim	Eventos do organizador autenticado
POST	/api/events	Sim	Cria novo evento
PUT	/api/events/{id}	Sim	Atualiza evento (so o organizador)
DELETE	/api/events/{id}	Sim	Remove evento (so o organizador)

7.3 Inscricoes, Comentarios e Reacoes

Metodo	Rota	Auth	Descricao
POST	/api/registrations	Sim	Inscreve usuario em evento
GET	/api/registrations/me	Sim	Minhas inscricoes
DELETE	/api/registrations/{id}	Sim	Cancela inscricao
GET	/api/comments?eventoId=X	Nao	Comentarios de um evento
POST	/api/comments	Sim	Adiciona comentario
DELETE	/api/comments/{id}	Sim	Remove comentario (so o autor)
GET	/api/reactions?eventoId=X	Nao	Reacoes de um evento
POST	/api/reactions	Sim	Adiciona reacao

Metodo	Rota	Auth	Descricao
DELETE	/api/reactions/{id}	Sim	Remove reacao (so o autor)

8 Camada de Infrastructure

8.1 DbContext e Configuracoes EF Core

O `AcademicEventsDbContext` configura os mapeamentos via *Fluent API* no metodo `OnModelCreating`:

- **Indice unico em User.Email**: impede dois usuarios com o mesmo email
- **Indice unico em (UsuarioId, EventoId) na Registration**: impede inscricao duplicada
- **Indice unico em (UsuarioId, EventoId) na Reaction**: impede reacao duplicada
- **DeleteBehavior.Restrict** em Event para User: impede deletar usuario que tem eventos
- **DeleteBehavior.Cascade** nos demais: ao remover usuario ou evento, registros dependentes sao apagados

8.2 Repository Pattern

Cada entidade tem seu proprio repository que abstrai o acesso ao banco:

```
// exemplo de consulta com Include para evitar N+1 queries
public async Task<List<Event>> GetAllAsync()
{
    return await _context.Events
        .Include(e => e.Organizador)
        .OrderByDescending(e => e.DataInicio)
        .ToListAsync();
}
```

O uso de `Include` garante que o nome do organizador seja carregado junto com o evento em uma unica query SQL, evitando o problema N+1.

8.3 Banco de Dados via Docker

O ambiente de banco de dados e provisionado via `docker-compose.yml`:

```
docker compose up -d # sobe o PostgreSQL 15 na porta 5432
```

A inicializacao do schema usa `EnsureCreated()` no startup da API, criando as tabelas automaticamente se nao existirem.

9 Regras de Negocio Implementadas

Regra	Implementacao
Email unico	Verificado no AuthService antes de inserir no banco (400 BadRequest)
Senha segura (min 6 chars)	Validacao de DataAnnotations no RegisterRequest
Credenciais invalidas	Mesma mensagem independente de email ou senha errados (seguranca)

Regra	Implementacao
DataFim maior que DataInicio	Validado no EventService ao criar evento (400 BadRequest)
So organizador edita evento	Verificacao de OrganizadorId no EventService (403 Forbidden)
Inscricao duplicada	Verificacao no RegistrationService antes do banco (400 BadRequest)
Reacao unica por evento	Verificacao no ReactionService antes do banco (400 BadRequest)
So autor deleta comentario	Verificacao de UsuarioId no CommentService (403 Forbidden)
Respostas HTTP corretas	200/201 sucesso, 400 dados invalidos, 401 sem auth, 403 sem permissao, 404 nao encontrado

10 Dificuldades e Decisoões Técnicas

10.1 Versoes dos Pacotes NuGet

A combinacao de versoes dos pacotes do Entity Framework Core para .NET 8 exigiu atencao especial. A versao 8.0.11 do `Npgsql.EntityFrameworkCore.PostgreSQL` foi escolhida por ser a mais estavel e compativel com o .NET 8 e o `Microsoft.EntityFrameworkCore.Design` 8.0.11.

10.2 Forbid() vs StatusCode(403)

O metodo `Forbid()` do `ControllerBase` do ASP.NET Core nao aceita uma mensagem de texto – ele espera um nome de esquema de autenticao. Para retornar 403 com mensagem de erro no corpo da resposta, o correto e usar `StatusCode(403, mensagem)`.

10.3 Application nao depende de Infrastructure

A camada Application so referencia o Domain (onde estao as interfaces de repository). A Infrastructure e registrada no DI pelo API via extension method, resolvendo as implementacoes concretas em tempo de execucao. Isso garante a inversao de dependencia correta.

10.4 Validacao em Dois Niveis

Para inscricoes e reacoes duplicadas, a validacao acontece tanto no service (verificacao previa antes do banco) quanto no banco (indice unico). O service lanca uma execucao customizada com mensagem amigavel; o banco e a ultima linha de defesa contra condicoes de corrida.

11 Como Executar o Projeto

- .NET 8 SDK
- Docker e Docker Compose
- (Opcional) dotnet-ef para migrations: `dotnet tool install --global dotnet-ef`

```
# 1. clonar o repositario
git clone https://github.com/JulianaBallin/Academic-Events-API.git
cd Academic-Events-API

# 2. subir o banco PostgreSQL via Docker
docker compose up -d
```

```
# 3. iniciar a API (o EnsureCreated cria as tabelas automaticamente)
cd AcademicEvents.API
dotnet run
```

Apos iniciar, acesse a documentacao interativa no Swagger em: <http://localhost:5000/swagger>

Para testar os endpoints com autenticao, clique em **Authorize** no Swagger e informe o token obtido no login no formato **Bearer {token}**.

12 Conclusao

A Academic Events API atende todos os requisitos obrigatorios do trabalho final: autenticao JWT, cinco entidades com relacionamentos, banco PostgreSQL via EF Core, arquitetura em cinco projetos separados, DTOs request/response, services com regras de negocio, repositories com repository pattern, Swagger/OpenAPI e versionamento via Git.

Alem dos requisitos minimos, o grupo implementou:

- Cinco execucoes customizadas com semantica clara de negocio
- Filtragem de eventos por status e por organizador
- Validacoes em duas camadas (service + banco) para evitar dados inconsistentes
- Documentacao XML em portugues em todos os arquivos
- Diagramas C4 nos quatro niveis (contexto, container, componente e codigo)
- Roteiro completo de testes manuais com cenarios de sucesso e de erro

O projeto demonstra dominio pratico dos conceitos de C# e ASP.NET Core abordados no modulo, resultando em uma API funcional, bem organizada e pronta para ser consumida por qualquer aplicacao cliente.